Язык программирования Python

Строки

Доцент кафедры ИВТ, к.т.н.
Проскурин Александр Викторович

Строки

Строка – базовый тип, представляющий собой **неизменяемую упорядоченную последовательность символов**. В Python острока определена как тип **str**.

Строки можно задать в программе с помощью строковых литералов. **Литералы** записываются с использованием **апострофов ' или кавычек "**:

```
>>> 'hello'
'hello'
>>> "Привет"
'Привет'
```

Внутри строки из кавычек могут быть апострофы.
Внутри строки из апострофов могут быть кавычки.

Без кавычек Python воспримет текст как имя переменной и попытается найти её содержимое:

```
>>> hello
Traceback (most recent call last):
  File "<pyshell#51>", line 1, in <module>
    hello
NameError: name 'hello' is not defined
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Многострочные блоки текста

Также можно использовать **апострофы и кавычки**, повторенные **трижды**. Главное достоинство такого задания строк в том, что их можно использовать для записи **многострочных блоков текста**.

Внутри такой строки возможно присутствие кавычек и апострофов, главное чтобы не было трёх кавычек подряд:

```
>>> """Текст,
содержащий кавычки "
и апострофы '
"""
'Текст,\nсодержащий кавычки "\nи апострофы \'\n'
```

В приведённом примере **\n** – это специальный символ, обозначающий перевод на новую строку. Помимо него в тексте также может использоваться символ табуляции **\t**, означающий отступ.

Символ **\'** – это **экранирование** апострофа, добавленное из-за того что при выводе строка обрамляется апострофами.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к символам. Индекс

Каждый символ строки имеет собственный порядковый номер (**индекс**). Нумерация начинается с нуля. Обратиться к отдельному символу можно записав имя строки и в квадратных скобках индекс символа:

```
>>> s = 'Пример!'
>>> s[0]
'П'
>>> s[2]
'и'
```

П	р	и	м	е	р	!
0	1	2	3	4	5	6

Для обращения к символам также могут применяться отрицательные индексы. В этом случае отсчёт ведётся с конца строки:

```
>>> s = 'Пример!'
>>> s[-1]
'!'
>>> s[-2]
'р'
```

П	р	и	м	е	р	!
-7	-6	-5	-4	-3	-2	-1

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к символам. Длина

Для вычисления индекса используется формула: `длина строки + отрицательный индекс`. Длину можно найти с помощью функции **len()**, которой передаётся строка в качестве аргумента:

```
>>> s = 'Пример!'
>>> len(s)
7
>>> s[len(s)-1]
'!'
```

П	р	и	м	е	р	!
0	1	2	3	4	5	6

Поскольку строка является **неизменяемой**, то присвоить новый символ по индексу не получится:

```
>>> s[0] = 'P'
Traceback (most recent call last):
  File "<pyshell#68>", line 1, in <module>
    s[0] = 'P'
TypeError: 'str' object does not support item assignment
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к символам. Срез (1)

Если вам необходимо изменить часть строки, то придётся создавать новую строку, используя имеющуюся. Для облегчения этой задачи применяются **срезы**:

```
>>> s = 'Питоны в Африке'
>>> s
'Питоны в Африке'
>>> s[1:4:1]
'ито'
```

П	и	т	о	н	ы		в		А	ф	р	и	к	е
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Срез – это специальная операция, позволяющая выбрать часть строки. При этом внутри квадратных скобок через двоеточие нужно указать три значения:

1. индекс начала (по умолчанию 0);
2. индекс конца (не включительно!) (по умолчанию последний);
3. шаг выборки (по умолчанию 1).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Обращение к символам. Срез (2)

При использовании среза не обязательно указывать все три параметра. Допустимо некоторые из них опускать.

Например:

П	и	т	о	н	ы		в		А	ф	р	и	к	е
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

```
>>> s = 'Питоны в Африке'
>>> s[2:5] # со 2-го индекса по 5-ый с шагом 1
'тон'
>>> s[:3] # с 0-го индекса по 3-ий с шагом 1
'Пит'
>>> s[:] # вся строка (создаётся копия)
'Питоны в Африке'
>>> s[::2] # с 0-го индекса до конца с шагом 2
'Птн  фие'
>>> s[::-1] # с конца до 0-го индекса с шагом -1 (переворот)
'екирфА в ынотиП'
>>> s = 'Три' + s[2:] # со 2-го индекса до конца с шагом 1
>>> s
'Тритоны в Африке'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Проверка наличия символа

Полезным является оператор **in**, позволяющий проверить вхождение в строку какого-либо символа:

```
>>> 'б' in 'Строка'
False
>>> 'а' in 'Строка'
True
```

Как правило он используется совместно с операторами if и while:

```
>>> if 'а' in 'кан':
        print('Символ есть в строке')
'Символ есть в строке'
>>> while input('Продолжать (y/n)?') in 'yY':
        print('Продолжаем работу цикла')
Продолжать (y/n)?y
Продолжаем работу цикла
Продолжать (y/n)?y
Продолжаем работу цикла
Продолжать (y/n)?n
```

На самом деле, мы выйдем из цикла при любом введённом символе, отличном от y или Y.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Операции над строками

С помощью операции **конкатенации** в Python можно объединить несколько строк в одну. В коде для этого используется символ «+»:

```
>>> 'Привет, ' + 'земляне!'
'Привет, земляне!'
>>> s = 'Здравствуйте, ' + 'земляне!'
>>> s
'Здравствуйте, земляне!'
```

Складывать можно только строки! Строку и, например, число сложить не получится.

Также можно **повторить** строку определённое количество раз. Для этого используется операция «*»:

```
>>> 'Строка' * 5
'СтрокаСтрокаСтрокаСтрокаСтрока'
>>> s = 2 * 'кан'
>>> s
'канкан'
```

На первом месте может быть и строка, и число.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (1)

При работе со строковым типом **str** часто используются **методы**. Это специальные встроенные функции, которые имеют две формы вызова: полную и сокращённую.

Полная форма вызова:

```
str.метод(строка, аргументы)
```

Аргументы могут отсутствовать.

Сокращённая форма вызова:

```
строка.метод(аргументы)
```

Например, вызовем метод upper(), который возвращает новую строку с символами исходной строки в верхнем регистре:

```
>>> s = 'Пример!'
>>> str.upper(s)
'ПРИМЕР!'
>>> s.upper()
'ПРИМЕР!'
```

Т.к. строка – неизменяемый тип данных, то метод upper() возвращает именно **новую строку**. Чтобы её сохранить, нужно присвоить результат переменной (можно той же самой).

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (2) is_()

Существуют методы проверки соответствия содержимого строки чему-либо. Все они возвращают True или False. Предположим, что **s** – это строка. Тогда:

Метод	Назначение
s.isalnum()	Возвращает True, если строка содержит только буквы и цифры и имеет ненулевую длину. Иначе – False
s.isalpha()	Возвращает True, если строка содержит только буквы и длина ненулевая
s.isdigit()	True, если содержит только цифры и длина ненулевая
s.islower()	Возвращает True, если все буквы строчные (маленькие), иначе – False
s.isupper()	Возвращает True, если все буквы прописные (большие), иначе – False
s.isspace()	Возвращает True, если строка состоит только из пробельных символов (пустая строка – False)

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (3) is_()

Методы можно вызывать как для переменных, так и для литералов:

```
>>> s = 'AxAxA'
>>> s.isalnum() # состоит ли из букв и цифр?
True
>>> s.isalpha() # состоит ли только из букв?
True
>>> s.isdigit() # состоит ли только из цифр?
False
>>> 'A'.isupper() # состоит ли из прописных
True
>>> '1'.isupper() # если в строке нет букв, то False
False
>>> '10'.islower() # если в строке нет букв, то False
False
>>> '\t \n'.isspace() # состоит ли из пробельных симв?
True
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (4) Преобразования

Существуют методы для изменения строки, путём преобразования её символов, замены или удаления. Все они возвращают новую строку. Предположим, что **s** – это строка. Тогда:

Метод	Назначение
s.lower()	Преобразование строки к нижнему регистру
s.upper()	Преобразование строки к верхнему регистру
s.capitalize()	Первый символ приводится к верхнему регистру, остальные к нижнему.
s.lstrip(***x***) s.rstrip(***x***) s.strip(***x***)	Удаляет символы, которые есть в строке ***x***, в начале / в конце / в начале и в конце строки **s**. Если не задать ***x***, то удаляет пробельные символы.
s.center(***w***)	Возвращает строку длинной ***w*** символов, в центре которой размещается строка **s**
s.replace(***old***, ***new***, ***n***)	Возвращает копию строки, в которой подстроки ***old*** заменены на ***new***. Если задан параметр ***n***, то заменяются только первые ***n*** вхождений

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (5) Преобразования

Методы можно вызывать последовательно друг за другом:

Сначала все пробелы заменяются на пустую строку (удаляются). Полученная строка проверяется на вхождение только букв.

```
>>> s = 'Привет всем'
>>> s.isalpha()
False
>>> s.replace(' ', '').isalpha()
True
>>> s.replace('ет', 'ат')
'Приват всем'
>>> s.lower() # приводим к нижнему регистру
'привет всем'
>>> 'гДЕ вСе'.capitalize() # приводим к обычному виду
'Где все'
>>> 'Ввели с интером\n'.strip() # удаляем пробелы
'Ввели с интером'
>>> 'Ввели с интером'.lstrip('вВм') # удаляем в, В, м
'ели с интером'
>>> '123'.center(11) # центрируем в строке длинной 11
'    123    '
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (6) Другие

Также имеются другие полезные методы. Предположим, что **s** – это строка. Тогда:

Метод	Назначение
s.split(***sep***, ***n***)	Разбивает строку **s** по подстроке ***sep*** и возвращает список полученных подстрок. Параметр ***n*** определяет максимальное количество разбиений (начиная слева)
sep.join(***lst***)	Объединяет строки из списка ***lst*** через разделитель **sep** в виде ***lst***[0] + **sep** + ***lst***[1] + **sep** + ***lst***[2] …
s.find(***sub***, ***start***, ***end***)	Возвращает наименьший индекс, с которого начинается вхождение подстроки ***sub*** в строку **s**. Поиск идёт в окне [***start***, ***end***). Если подстрока не найдена, возвращается -1.
s.count(***sub***)	Возвращает количество вхождений подстроки ***sub*** в строку **s**.
s.startswith(***prefix***)	Проверяет, начинается ли строка на ***prefix*** (возвращает True или False)
s.endswith(***suffix***)	Проверяет, оканчивается ли строка на ***suffix*** (возвращает True или False)

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (7) Другие

Методы можно вызывать последовательно друг за другом:

```
>>> s = 'канкан'
>>> s.count('ан') # количество вхождений ан
2
>>> s.startswith('ка') # начинается ли с ка?
True
>>> s.endswith('ет') # заканчивается ли на ет?
False
>>> 'ОхОхОхАхИх'.split('х') # разбиваем по х
['О', 'О', 'О', 'А', 'И', '']
>>> 'к'.join(_) # объединяем список с прошлой команды
'ОкОкОкАкИк'
>>> 'Ввели с интером'.find('и') # ищем первую и
4
>>> 'Ввели с интером'.find('и', 5) # ищем вторую и
8
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Методы строк (8) find()

Полезным является метод **find(sub, start, end)**, который возвращает индекс первого вхождения подстроки sub. Поиск идёт в диапазоне индексов [start, end). По умолчанию ведётся поиск с начала строки и до конца. Если подстрока не найдена, то возвращается -1.

```
>>> s = 'AxAxA'
>>> s.find('A')
0
>>> s.find('A')  # при повторе результат не меняется
0
```

Для нахождения всех вхождений подстроки необходим цикл:

```
>>> idx = s.find('A')  # находим первое вхождение
>>> while idx != -1:   # если подстрока найдена
        print(idx)     # то выводим текущий результат
        idx = s.find('A', idx+1) # и ищем следующий
0
2
4
```

После первого вхождения продолжаем поиск со следующего символа.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример преобразования строки в список

Необходимо с клавиатуры ввести элементы числового списка. При этом элементы списка, разделённые запятыми, вводятся за один раз.

```
def str_to_ls(st):
  st = st.replace(' ', '') # удаляем все пробелы
  print('Строка без пробелов:', st)
  ls = st.split(',') # разделяем на список по запятым
  print('Список строк:', ls)
  ls = [float(i) for i in ls] # формируем список чисел
  return ls

s = input('Введите числа через запятую: ')
ls = str_to_ls(s) # преобразуем в список чисел
print('Список чисел:', ls)
```

Пример:

```
Введите числа через запятую: 5,87, 8.45,8  ,7
Строка без пробелов: 5,87,8.45,8,7
Список строк: ['5', '87', '8.45', '8', '7']
Список чисел: [5.0, 87.0, 8.45, 8.0, 7.0]
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример преобразования списка в строку

Если стоит обратная задача – преобразовать существующий список в одну большую строку, то необходимо перевести каждый элемент списка в строку и воспользоваться методом join().

```
def ls_to_st(ls):
  ls = [str(i) for i in ls] # формируем список строк
  # можно преобразовать так: ls = list(map(str, ls))
  print('Список строк:', ls)
  st = ', '.join(ls) # формируем одну строку
  return st

ls = [1, 5, 4, 'Строка']
s = ls_to_st(ls) # преобразуем в строку
print('Строка:', s)
```

map() применяет функцию (первый параметр) ко всем элементам списка.

Пример:

```
Исходный список: [1, 5, 4, 'Строка']
Список строк: ['1', '5', '4', 'Строка']
Строка: 1, 5, 4, Строка
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

F-строки

F-строки предоставляют способ встраивания выражений внутри строковых литералов.

Стоит обратить внимание на то, что f-строка является выражением, которое оценивается по мере выполнения, а не постоянным значением.

В исходном коде Python **f-строка** является **литеральной строкой с префиксом f**, которая содержит **выражения внутри фигурных** скобок. Выражения заменяются их значениями.

```
1  >>> name = 'Виктор'
2  >>> age = 30
3  >>> print(f'Здравствуй, {name}. Тебе {age} лет.')
4  Здравствуй, Виктор. Тебе 30 лет.
5  >>> stock = ['Газеты', 'Конверты', 'Блокноты', 'Ручки']
6  >>> #Можно написать многострочные f-строки внутри круглых скобок
7  ... message = ( f'У нас в продаже {stock[1]} и {stock[2]}, '
8  ... f'а также {stock[3].lower()}.' )
9  >>> print(message)
10 У нас в продаже Конверты и Блокноты, а также ручки.
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Форматированный вывод (1)

Иногда желательно отформатировать выводимую на экран информацию для удобного восприятия человеком. Для этого в f-строках используются **спецификаторы формата**, которые указываются **внутри фигурных** скобок после **выводимого выражения** и символа **двоеточия «:»**.

Общий вид спецификаторов формата (каждый из них можно опустить) следующий:

заполнитель	выравнивание	знак	#	0	ширина	.точность	тип
любой символ кроме }	'<' '>' '^' '='	числа: '+' '-' ' '	целые числа: '#0b' '#0o' '#0x' '#0X'	дополнение чисел нулями	минимальная ширина поля вывода	числа: точность строки: максимальная ширина вывода	целые числа: 'b' 'c' 'd' 'n' 'o' 'x' 'X' вещественные числа: 'e' 'E' 'f' 'F' 'g' 'G' 'n' '%'

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Форматированный вывод (2) Выравнивание

Первый набор опций используется **совместно с заданием ширины** для вывода значения.

Опция	Значение
'<'	Выравнивает по левому краю (используется по умолчанию)
'>'	Выравнивает по правому краю
'^'	Выравнивает по центру
'='	Только для чисел. Вставляет заполнитель между знаком и числом

```
>>> var = 'привет!'
>>> f'{var:<11}'  # Слева в поле шириной 11 символов
'привет!    '
>>> f'{var:>11}'  # Справа в поле шириной 11 символов
'    привет!'
>>> f'{var:^11}'  # По центру в поле шириной 11 симв.
'  привет!  '
>>> f'{-1234:*=10}'
'-*****1234'
```

* – заполнитель;
= – выравнивание;
10 – ширина вывода.

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Форматированный вывод (3) Знак

Данная опция применима только к числам.

Опция	Значение
'+'	Знак должен отображаться как для отрицательных, так и для положительных чисел
'-'	Знак отображается только для отрицательных чисел (используется по умолчанию)
' '	Выводит пробел для положительных чисел и минус для отрицательных

```
>>> f'{54:+} {-54:+}'  # Отображать знак всегда
'+54 -54'
>>> f'{5.2:-} {-14}'  # Знак только у отрицательных
'5.2 -14'
>>> f'{4: } {-14.1: }'  # Пробел или минус
' 4 -14.1'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Форматированный вывод (4)

Данная опция применима только к целым числам.

Опция	Значение
'#0b'	Выводит число в двоичной системе счисления
'#0o'	Выводит число в восьмеричной системе счисления
'#0x'	Выводит число в шестнадцатеричной системе счисления. При этом буквы в нижнем регистре
'#0X'	Выводит число в шестнадцатеричной системе счисления. При этом буквы в верхнем регистре

```
>>> f'{15} {15:#0b}'  # 15 в двоичной
'15 0b1111'
>>> f'{15} {15:#0o}'  # 15 в восьмеричной
'15 0o17'
>>> f'{15} {15:#0x}'  # 15 в шестнадцатеричной
'15 0xf'
>>> f'{15} {15:#0X}'  # 15 в шестнадцатеричной
'15 0XF'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Форматированный вывод (5) 0, ширина, .

Ширина задаёт минимальный размер поля, в которое будет выводится значение. При этом для чисел можно указать заполнение свободного пространства нулями:

```
>>> f'{15:0=10}'  # через выравнивание
'0000000015'
>>> f'{15:010}'  # через ноль
'0000000015'
```

Точность используется совместно с вещественными числами, ограничивая количество цифр после запятой:

```
>>> f'{1/6} {1/6:.2}'  # 1/6 – вычисляемое выражение
'0.1666666666666666 0.17'
```

Для строк точность ограничивает максимальный размер:

```
>>> s = 'Пример строки' # 13 символов
>>> f'{s:10}'   # задание ширины не ограничивает сверху
'Пример строки'
>>> f'{s:.10}'  # а точность ограничивает
'Пример стр'
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Форматированный вывод (6) Тип (1)

Тип влияет на способ вывода чисел.

Опция	Значение
'b'	Выводит число в двоичной системе счисления
'd' 'n'	Выводит число в десятичной системе счисления
'o'	Выводит число в восьмеричной системе счисления
'x' 'X'	Выводит число в шестнадцатеричной системе счисления. В нижнем или в верхнем регистре

```
>>> f'{15:d} {15:b}'  # 15 в двоичной
'15 1111'
>>> f'{15:n} {15:o}'  # 15 в восьмеричной
'15 17'
>>> f'{15} {15:x}'  # 15 в шестнадцатеричной
'15 f'
>>> f'{15} {15:X}'  # 15 в шестнадцатеричной
'15 F'
```

В отличие от # здесь не выводятся префиксы, указывающие на систему счисления

© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Форматированный вывод (7) Тип (2)

Тип влияет на способ вывода чисел.

Опция	Значение
'e' 'E'	Научный формат – выводит с экспонентой
'f' 'F'	Формат с фиксированной точкой
'g' 'G' 'n'	Общий формат – в зависимости от размера числа может выводить в научном или фиксированном формате
'%'	Умножает на 100 и выводит с символом процента

```
>>> f'{15.0123456789:.4e}'  # научный
1.5012e+01
>>> f'{15.0123456789:.4f}'  # фиксированный
15.0123
>>> f'{15.0123456789:.4g}'  # общий
15.01
>>> f'{0123456789.15:.4g}'  # общий
1.235e+08
>>> f'{0.15:.0%}'  # процент
15%
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева

Пример форматированного вывода

Выведем на экран в виде таблицы информацию о стране, столице и количестве населения.

```
title = ('Страна', 'Столица', 'Население')
countries = (('КНР', 'Пекин', 1400970200),
             ('США', 'Вашингтон', 329210630),
             ('РФ', 'Москва', 146877088))
width =(8, 11, 11) # Хранит ширину столбцов
# Выводим горизонтальную линию во всю длину таблицы
# (ширина всех столбцов плюс промежутки между ними)
print( '-' * (sum(width) + 10) )
# Выводим заголовок таблицы
print((f'| {title[0]:-^{width[0]}} | {title[1]:-^{width[1]}} '
       f'| {title[2]:-^{width[2]}} |'))
# Выводим отдельно каждую страну
for cnt in countries:
    print( '-' * (sum(width) + 10) )
    print((f'| {cnt[0]:<{width[0]}} '
           f'| {cnt[1]:<{width[1]}} '
           f'| {cnt[2]:>{width[2]}} |'))
# Закрываем таблицу вертикальной линией
print( '-' * (sum(width) + 10) )
```

- – заполнитель
^ – по центру
{width[1]} – ширина

```
---------------------------------------
| -Страна- | --Столица-- | -Население- |
---------------------------------------
| КНР      | Пекин       |  1400970200 |
---------------------------------------
| США      | Вашингтон   |   329210630 |
---------------------------------------
| РФ       | Москва      |   146877088 |
---------------------------------------
```


© Проскурин Александр Викторович, к.т.н., доцент каф. ИВТ СибГУ им. М.Ф. Решетнева